# DigiPen Institute of Technology

# CS 372/573: Recommender System

## Objectives

By completing this assignment, you will:

1. Implement user-based collaborative filtering with KNN as base
2. Build matrix factorization models using SVD
3. Create knowledge-based recommender using user preferences
4. Integrate approaches into a hybrid recommender system
5. Evaluate system performance using multiple metrics

## Dataset

- Download MovieLens 100K from GroupLens
- Use u1.base (training) and u1.test (testing) files
- 100K ratings, 943 users, 1,682 movies

## Implementation Instructions

### Part 1: Data Setup and User-Based Collaborative Filtering (CF) (50 points)

1. **Load and preprocess data:**
    - Create user-item rating matrix.
    - Analyze sparsity:
        - calculate percentage of missing ratings: Sparsity = (Number of Missing Ratings / Total Possible Ratings) × 100
    - Explain how to Handle missing rating
    - Similarities on co-rated items only
    - Compute per-user mean rating on train only for later imputation/centering
    - Consider min overlap on co-rated Movies

2. **Model 1: Build K-Nearest Neighbors recommender (baseline):**
    - Find k most similar users for any target user (test different values of k)

- Only consider users with positive similarity scores
- Predict ratings using weighted average (check slides for the formula)
- Handle edge case (Check Class notes)

3. **Model 2: Build cluster-based KNN recommender**
    - **Apply K-means clustering to users**
        1. Cluster users based on their rating vectors using K-means
        2. Use K-means++ to find the initial centroids of clusters
        3. Choose the first centroid randomly
        4. Clearly explain how you chose the number of clusters and why you think it will work?
    - Assign each user to their closest cluster
    - For target users, identify their cluster
    - Use other users in same cluster as neighbors for prediction
    - Predict ratings
    - Fallback plan: if <k valid neighbors or no raters for item

4. **Evaluate CF performance:**
    - Calculate RMSE on test set for each model and k value combination
    - Create line plot: number of clusters on x-axis, RMSE on y-axis
    - Create results table showing best performing configuration

**Part 2: Matrix Factorization (25 points)**

1. **Implement SVD-based matrix factorization:**
    - Use scipy.sparse.linalg.svds or sklearn.decomposition.TruncatedSVD
    - Choose the factors to test with
    - Reconstruct matrix

2. **Compare matrix factorization methods:**
    - Plot RMSE vs number of factors for SVD
    - Compare the results with best model from part 1

- Document computational time
- Identify optimal dimensionality for each method

**Part 3: Knowledge-Based (KB) Recommender (25 points)**

1. **Create preference elicitation interface:**
    - Collect user preferences for:
        - Genres: Multi-select from MovieLens categories (Action, Comedy, Drama, etc.)
        - Release years: Min and max year range
        - Rating threshold: Minimum acceptable average rating (1-5)
        - Runtime: Short (<90min), Medium (90-120min), Long (>120min), Any

2. **Parse movie metadata:**
    - Extract genres from movie data
    - Parse release years from movie titles
    - Calculate average ratings for each movie from training data
    - Handle missing runtime data

3. **Implement content-based scoring:**
    - **Genre matching (50% weight):** Calculate overlap using set intersection:
        - score = len(movie_genres ∩ preferred_genres) / len(preferred_genres) * 40
    - **Year preference (30% weight)**
    - **Rating filter (20% weight)**
    - Total score = sum of all components (max 100)

4. **Generate recommendations with explanations:**
    - Create 3 different user preference profiles
    - Return top 3 movies with explanations of why each was selected
        - Example: "Recommended because: matches Action+Sci-Fi genres, 1999 release in your range, 4.3/5 rating above threshold"

**Part 4: Hybrid System Integration (Be creative! Below are only suggestions) (25 points)**

1. **Implement decision logic:**
    - New users (no ratings): Use knowledge-based only
    - Users with <5 ratings: Knowledge-based primary (80%) + CF supplement (20%)
    - Users with 5-20 ratings: Balanced hybrid (CF 60% + KB 40%)
    - Users with >20 ratings: CF primary (CF 80% + KB 20%)

2. **Create score combination function:**
    - Normalize CF predictions to 0-100 scale to match KB scores
    - Weight and combine scores based on user's rating history
    - Handle cases where one method fails (fallback to other method)
    - Ensure system always returns recommendations

**Part 5: System Evaluation (25 points)**

- **Create comprehensive analysis**
    - Summarize performance of all methods
    - Analyze when each approach works best
    - Identify challenges encountered
    - Discuss scalability
    - Suggest improvements

**Deliverables**

**Jupyter Notebook containing:**

- Complete implementation of all components
- Performance comparison table
- Analysis of when each method works best
- Code documentation with clear explanations
- Results visualization and interpretation

**Key Implementation Notes**

- Test edge cases (empty clusters, users not fitting well in any cluster)
- Use appropriate data structures for sparse matrices
- Document your design decisions

- Include error handling for robustness

**Resources**

- **Documentation:** Pandas | SciPy | Scikit-learn
- **Code Examples:** Hands-On Recommendation Systems with Python
- **Dataset Info:** MovieLens README